Lezione 8 – La classe `Object`, `clone` e le relazioni tra classi

`toString`, `equals` e `hashCode`, `instanceof`, copia di oggetti, contenimento fra classi

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

28 ottobre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 7** (*15 min*)
   - uno di voi alla lavagna con `Triangolo`, `ContoRisparmio` o `Borsista`: lo commentiamo insieme
2. **La superclasse** `Object` (*25 min*)
   - `toString`; `equals(Object)` e `hashCode`; `getClass` e `instanceof`
   - copiare un oggetto: `clone`, copia superficiale e copia profonda
3. **Relazioni tra classi** (*20 min*)
   - contenimento stretto e lasco: composizione e aggregazione; dipendenza
   - ereditarietà o contenimento?
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: `Complex` – `toString`, `equals` e `hashCode` fatti bene
   - *Insieme, passo passo*: `Persona` e `Indirizzo` – la copia profonda
5. **Wrap-up e Homework 8** (*10 min*)

La superclasse `Object` – `ProvaObject.java`

La classe `Object`

Una classe senza `extends` estende `java.lang.Object`: ogni classe ne discende e ne **eredita i metodi**. Un riferimento di tipo `Object` può portare a un oggetto di qualunque classe.

I metodi che useremo, nella versione di `Object`:

Metodo	**Che cosa fa in `Object`**
`String toString()`	nome della classe, `@`, codice hash
`boolean equals(Object obj)`	confronta i riferimenti, come `==`
`int hashCode()`	un intero associato all'oggetto
`Class getClass()`	la classe dell'oggetto (`final`: non ridefinibile)
`Object clone()`	copia campo per campo (`protected`: non visibile ai client)

Gli altri (`wait`, `notify`, `finalize`) non li useremo.

```
Persona p = new Persona("Anna", "Russo");
Persona q = new Persona("Anna", "Russo");

System.out.println(p);                  // Object
System.out.println(p.descrizione());
System.out.println(p.hashCode()); // Object
System.out.println(p.getClass()); // Object
```

```
Persona@2a139a55
Anna Russo
705927765
class Persona
```

- `Persona` (versione 3) è quella della lezione 7: non dichiara `toString`, `hashCode` né `getClass`, ma le chiamate compilano: li eredita da `Object`
- `Object` non sa nulla di nome e cognome: il suo `toString` stampa classe e codice hash in esadecimale (`2a139a55` = 705927765; il numero cambia da una macchina all'altra). Per avere versioni sensate, la classe li **ridefinisce**: è il lavoro di oggi

toString: Java lo chiama da solo – ProvaToString.java

In Persona (versione 4) e in Studente:

```
    @Override
    public String toString() {
        return nome + " " + cognome;
    }
```

```
    @Override
    public String toString() {
        return super.toString()
                + ", matricola " + matricola;
    }
```

Nel main:

```
System.out.println(p);                  // p.toString()
System.out.println(s);
System.out.println("Ciao, " + p);      // p.toString()

int[] v = { 1, 2, 3 };
System.out.println(v);                  // Object!
System.out.println(Arrays.toString(v));
```

toString

La rappresentazione testuale dell'oggetto. Java lo chiama **da solo** ogni volta che al posto di un oggetto serve una String: in println(x), in "..." + x, in String.valueOf(x).

```
Anna Russo
Luca Bianchi, matricola N86001234
Ciao, Anna Russo
[I@1d44bcfa
[1, 2, 3]
```

- finora l'abbiamo chiamato descrizione(): con il nome toString basta println(p). Da oggi la descrizione si chiama così, con @Override
- vale per ogni oggetto, anche per gli **array**, che però non lo ridefiniscono: println(v) stampa [I@1d44bcfa ([I: array di int); per questo serve Arrays.toString (lezione 4)

equals: perché ridefinire equals(Object) – ProvaObject.java

In Persona (versione 3), nello stile della lezione 6:

```
    public boolean equals(Persona altra) {
        return nome.equals(altra.nome)
                && cognome.equals(altra.cognome);
    }
```

Nel main, con p e q entrambi "Anna Russo":

```
System.out.println(p == q);
System.out.println(p.equals(q));  // overload
Object o = q;
System.out.println(p.equals(o));  // Object!
Persona[] a = { p };
Persona[] b = { q };
System.out.println(Arrays.equals(a, b));
```

```
false
true
false
false
```

p.equals(o) e Arrays.equals dicono false, eppure o porta a q, e gli array contengono persone uguali. È l'equals di Object, quello che nella lezione 6 faceva compilare c.equals(d).

equals(Object)

La versione di Object confronta i riferimenti, come ==. Una classe che vuole confrontare il *contenuto* la **ridefinisce**, con la stessa firma: public boolean equals(Object obj).

- equals(Persona) è un **overload**: Persona ha due equals, il nostro e quello di Object. Fra i due sceglie il **compilatore**, dal tipo *dichiarato* dell'argomento (lezione 7): Persona per q, Object per o
- Arrays.equals lavora su Object[]: dentro, a[0].equals(b[0]) ha l'argomento di tipo Object, quindi è l'equals di Object: p e q sono oggetti distinti, false. Così le collezioni (lezione 12): la libreria chiama **sempre** equals(Object)

equals(Object): come si scrive – Persona.java (versione 4)

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    Persona altra = (Persona) obj;
    return nome.equals(altra.nome)
            && cognome.equals(altra.cognome);
}
```

Lo schema è sempre lo stesso, per ogni classe: cambiano solo il cast e l'ultimo confronto. `altra.nome` è `private`, ma siamo dentro `Persona`: visibile per ogni oggetto della classe (lezione 5).

Il parametro è `Object`: `p.equals(x)` compila con `x` di **qualunque** tipo, anche `"Anna"` o `null`. Per questo il metodo controlla prima che cosa ha ricevuto, e solo dopo ci guarda dentro.

I quattro passi

1. **stesso oggetto**: `this == obj`, uguale di sicuro: `true` senza confrontare nulla
2. `null`: `false`, e si esce prima di chiamare qualcosa su `obj`, che terminerebbe il programma con un errore a tempo di esecuzione
3. **classe diversa**: `false`. `getClass()` è `this.getClass()`: la classe dell'oggetto, non del riferimento (fra due slide)
4. **cast e confronto**: ora `obj` porta di sicuro a una `Persona` e il downcasting (lezione 7) non può fallire; serve per leggere `altra.nome`, che `Object` non ha. Poi campo per campo: `equals` per le `String` e gli altri riferimenti, `==` per i primitivi

equals e hashCode: la regola – Studente.java e ProvaEquals.java

In Persona:

```
@Override
public int hashCode() {
    return Objects.hash(nome, cognome);
}
```

In Studente:

```
@Override
public boolean equals(Object obj) {
    if (!super.equals(obj)) {
        return false;
    }
    Studente s = (Studente) obj;
    return matricola.equals(s.matricola);
}

@Override
public int hashCode() {
    return Objects.hash(super.hashCode(),
            matricola);
}
```

`super.equals`: passi 1–3, nome e cognome; poi il cast è sicuro.

Funzione di hash e regola di hashCode

`hashCode` è una *funzione di hash*: da un oggetto calcola un `int` che lo riassume, l'*hash*. Serve alle strutture dati che cercano un oggetto in base al suo hash invece di confrontarlo con tutti gli altri (lezione 12). Perché funzionino: se `a.equals(b)` è `true`, allora `a.hashCode() == b.hashCode()`. Chi ridefinisce `equals` ridefinisce **anche** `hashCode`, sugli stessi campi.

```
System.out.println(p.equals(s)); // Studente
System.out.println(p.hashCode()
        == q.hashCode());
```

```
false
true
```

Ora anche `p.equals(o)` e `Arrays.equals(a, b)` danno `true`. `Objects.hash` (`java.util`) combina gli hash degli argomenti: valori uguali, hash uguale.

getClass e instanceof – ProvaInstanceof.java

```java
/** LEGGE x: se porta a uno studente,
 *  ne stampa la matricola. */
public static void stampaMatricola(Persona x) {
    if (x instanceof Studente) {
        Studente s = (Studente) x;  // sicuro
        System.out.println(s.getMatricola());
    } else {
        System.out.println("nessuna");
    }
}
```

Nel main:

```java
Persona p = new Persona("Anna", "Russo");
Persona q = new Studente("Luca", "Bianchi",
        "N86001234");
System.out.println(p.getClass());
System.out.println(q.getClass());
System.out.println(q instanceof Studente);
System.out.println(q instanceof Persona);
stampaMatricola(p);
stampaMatricola(q);
```

instanceof

`x instanceof C`: true se l'oggetto a cui `x` porta è di classe `C` **o di una sua sottoclasse**; con `x` a `null`, `false`.

```
class Persona
class Studente
true
true
nessuna
N86001234
```

- `getClass()` dà la classe **esatta** dell'oggetto (`Studente` per `q`); `instanceof` è vero anche per le superclassi. In `equals` serve `getClass`: con `instanceof`, `p.equals(s)` sarebbe vero e `s.equals(p)` no: uguaglianza non simmetrica
- è l'operatore promesso nella lezione 7: controllo *prima* del cast, che così non fallisce. Serve per i metodi della sola sottoclasse, non per quelli ridefiniti

Copiare un oggetto: `clone` – `ProvaClone.java`

```
public class Persona implements Cloneable {

    @Override
    public Persona clone()
            throws CloneNotSupportedException {
        return (Persona) super.clone();
    }
```

Nel `main` (dichiarato `throws`):

```
Persona p = new Persona("Anna", "Russo");
Persona q = p.clone();
System.out.println(p == q);
System.out.println(p.equals(q));
int[] v = { 1, 2, 3 };
int[] w = v.clone();   // array: clonabile
w[0] = 9;
System.out.println(Arrays.toString(v));
```

```
false
true
[1, 2, 3]
```

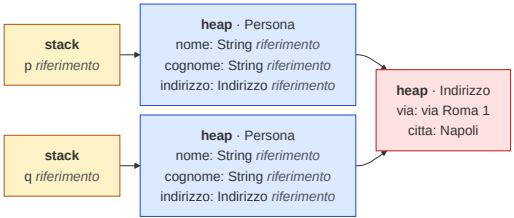
clone

`Object.clone()` crea un **nuovo oggetto della stessa classe** e vi copia, una per una, le variabili di istanza: una *copia superficiale*.

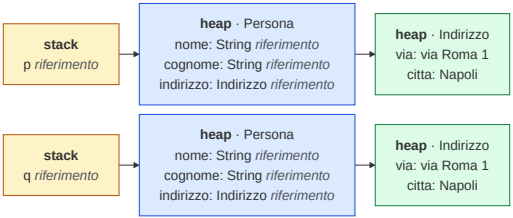
- due righe che oggi **copiamo e basta**: `implements Cloneable`, un'*interfaccia* (lezione 9) che autorizza la copia, e `throws CloneNotSupportedException`, un'*eccezione* (lezione 10), su `clone` e, fino a quella lezione, su chi lo chiama
- `super.clone()` fa il lavoro e restituisce un `Object`: da qui il cast. Il risultato `Persona` è lecito: nella ridefinizione si ammette un tipo **più specifico**
- gli **array** sono già clonabili, senza `throws`: `v.clone()` come `Arrays.copyOf(v, v.length)`

Copia superficiale e copia profonda

superficiale: `q` copia di `p`, ma l'`Indirizzo` è condiviso



profonda: anche l'`Indirizzo` è copiato



Copia superficiale, copia profonda

La copia *superficiale* copia le variabili di istanza: dei riferimenti copia il riferimento, e i due oggetti **condividono** gli oggetti interni. La copia *profonda* copia anche gli oggetti interni (in pratica quelli modificabili: gli immutabili, come le `String`, si possono condividere).

Nell'esercitazione `Persona` avrà, oltre a nome e cognome (`String`), un `indirizzo`: non una `String`, ma un oggetto `Indirizzo` con via e città, e la città può cambiare. Con la copia superficiale `q.trasferisci("Milano")` cambierebbe la città anche di `p`: è il privacy leak (lezione 6), con lo stesso rimedio, copiare l'oggetto interno. In `clone`: prima `super.clone()`, poi `c.indirizzo = indirizzo.clone()`.

Costruttore di copia o `clone`? – `ProvaClone.java`

	Costruttore di copia (L5)	`clone` **(oggi)**
si scrive	`Persona(Persona p)`	`implements Cloneable;` `clone()` con `super.clone()`
si usa	`new Persona(p)`	`p.clone()`
classe della copia	`Persona`: la decide il chiamante	quella dell'**oggetto**: da uno `Studente`, uno `Studente`
controlli sui dati	sì, nel costruttore	no: nessun costruttore chiamato
costo	nessuno	`Cloneable`, `throws`, cast

- `q = p` copia solo il riferimento (lezione 6): per copiare lo *stato* serve uno dei due
- di norma il **costruttore di copia**: più semplice e sotto controllo; `clone` quando la classe lo offre già (array, collezioni della lezione 12) o in una gerarchia, dove dà la copia della classe giusta

Nel `main` di `ProvaClone`:

```
Persona x = new Studente("Luca",
        "Bianchi", "N86001234");
Persona y = x.clone();    // ereditato
System.out.println(y.getClass());
System.out.println(y);
```

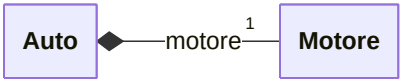
```
class Studente
Luca Bianchi, matricola N86001234
```

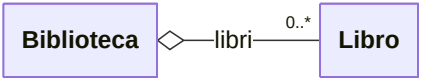
`Studente` non ridefinisce `clone`: eredita quello di `Persona`, che chiama `super.clone()`. `Object.clone` crea un oggetto della classe **dell'oggetto**, non del riferimento: `y` porta a uno `Studente` completo di matricola. Se `Persona` avesse il costruttore di copia della lezione 5, `new Persona(x)` darebbe una `Persona`, senza matricola.

Relazioni tra classi: contenimento (*ha-un*)

Contenimento

Un oggetto, il *contenitore*, ne contiene un altro, che ne è una parte: un'auto **ha un** motore, una biblioteca ha dei libri. In Java il contenitore ha una variabile di istanza di tipo riferimento che porta all'oggetto contenuto.





- **contenimento stretto** (*composizione*): l'oggetto contenuto non ha vita propria: nasce e muore con il contenitore, che lo **crea** nel costruttore e non lo cede mai all'esterno. Un'auto e il suo motore
- **contenimento lasco** (*aggregazione*): l'oggetto contenuto esiste per conto suo: lo crea il client e lo **passa** al contenitore (al costruttore o a un metodo), che ne memorizza il riferimento. Può stare in più contenitori e sopravvivere loro. Una biblioteca e i suoi libri

È la composizione della lezione 5, vista più da vicino: lì il rombo pieno; per l'aggregazione il rombo è **vuoto**. Il rombo sta dal lato del contenitore; sull'altro lato, la molteplicità.

	composizione	**aggregazione**
chi crea la parte	il contenitore	il client
la parte è condivisa?	no	può esserlo
se il contenitore muore	muore anche lei	resta

Composizione: Auto e Motore – Auto.java

```java
/** Un'auto HA UN motore: composizione. */
public class Auto {

    private final String targa;
    private final Motore motore;   // creato qui

    public Auto(String targa, int cilindrata) {
        this.targa = targa;
        motore = new Motore(cilindrata);
    }

    public void accendi() {
        motore.avvia();
    }

    @Override
    public String toString() {
        return "auto " + targa
                + ", motore " + motore;
    }
```

Non mostrati: `getTarga`; `isAccesa` e `spegni`, che delegano anch'essi al motore.

In `ProvaRelazioni`:

```java
Auto a = new Auto("AB123CD", 1200);
a.accendi();
System.out.println(a);
```

```
auto AB123CD, motore 1200 cc, acceso
```

- il client passa la **cilindrata**, non un motore: è `Auto` a fare `new Motore(...)`. `Motore` (acceso o spento, `avvia()`, `spegni()`) non sa di stare in un'auto
- nessun `getMotore`: il motore non esce dall'auto, nessun alias. I client parlano con l'auto (`accendi`), che **delega** al motore (`motore.avvia()`)
- `", motore " + motore` chiama il `toString` di `Motore`: un `toString` ne usa un altro
- quando l'auto diventa orfana, lo diventa anche il motore: il garbage collector (lezione 5) li elimina insieme. Lo abbiamo già fatto ogni volta che un costruttore crea con `new` una variabile di istanza

Aggregazione: Biblioteca e Libro – Biblioteca.java

```java
/** Biblioteca HA DEI libri: aggregazione. */
public class Biblioteca {

    private final String nome;
    private final Libro[] libri;    // ricevuti
    private int quanti;

    public Biblioteca(String nome, int max) {
        this.nome = nome;
        libri = new Libro[max];
        quanti = 0;
    }

    /** MODIFICA: false se e' piena. */
    public boolean aggiungi(Libro l) {
        if (quanti == libri.length) {
            return false;
        }
        libri[quanti] = l;
        quanti++;
        return true;
    }
```

Non mostrati: `getQuanti` e `toString`, che elenca i libri.

In `ProvaRelazioni`:

```java
Libro l1 = new Libro("Programmare in Java",
        "Deitel");
Libro l2 = new Libro("Effective Java",
        "Bloch");
Biblioteca b1 = new Biblioteca("Civica", 2);
Biblioteca b2 = new Biblioteca("DIETI", 10);
b1.aggiungi(l1);
b1.aggiungi(l2);
b2.aggiungi(l1);
System.out.println(b1);
```

```
Civica, libri: 2
  Programmare in Java (Deitel)
  Effective Java (Bloch)
```

- i libri nascono nel client, **prima** della biblioteca, e le vengono passati: `aggiungi` memorizza il riferimento. L'array, creato qui, è composizione; i libri, aggregazione (`0..*`)
- `l1` sta in due biblioteche: un **alias voluto**, non un privacy leak: `Libro` è immutabile. Se `b1` diventa orfana, `l1` resta: `b2` e il client lo riferiscono

Dipendenza; ereditarietà o contenimento?

Dipendenza

`A` dipende da `B` se un metodo di `A` **crea** un oggetto `B`, **chiama** un metodo su un `B` o **riceve** un `B` come parametro. Se `B` cambia interfaccia, `A` va ricontrollata.

Il contenimento è una dipendenza *stabile* (una variabile di istanza); le altre durano quanto il metodo. Oggi:

- `ProvaRelazioni` **crea** auto, libri e biblioteche
- `Biblioteca.aggiungi(Libro l)` **riceve** un libro
- `Auto.accendi()` **chiama** `motore.avvia()`

In UML la dipendenza generica è una freccia tratteggiata da `A` a `B`; il contenimento ha i rombi visti.

Ereditarietà o contenimento? Entrambe riusano una classe esistente.

- la domanda: **è-un** o **ha-un**? Uno `Studente` è una `Persona`; un'`Auto` *ha* un `Motore`, non è un motore
- la prova: i metodi `public` della classe esistente devono far parte dell'interfaccia della nuova? `getNome()` su uno studente ha senso: ereditarietà. `avvia()` su un'auto no: l'auto offre `accendi()`, e il motore resta dentro. Con `Auto extends Motore`, ogni auto avrebbe `avvia()` e `spegni()` nell'interfaccia
- l'**accoppiamento** (lezione 4): il contenimento, e l'aggregazione in particolare, accoppia meno: il contenitore usa solo l'interfaccia della classe contenuta, che può cambiare dentro senza conseguenze. La sottoclasse dipende dalla superclasse molto di più: costruttori da chiamare, metodi da ridefinire
- nel dubbio, **contenimento**: più facile da modificare. L'ereditarietà quando l'è-un è vero e serve il polimorfismo

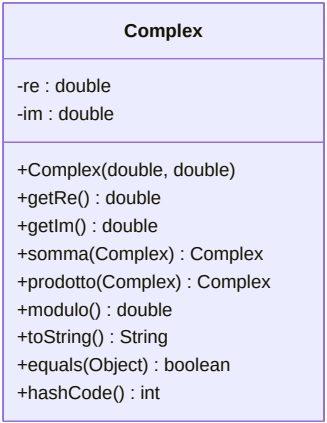
Commentiamo insieme (20 min) – il problema

Il problema. Un programma di calcolo lavora con numeri complessi $z = a + b\,i$: li somma, li moltiplica, ne calcola il modulo e li stampa. Soprattutto deve **confrontarli**: due numeri con la stessa parte reale e la stessa parte immaginaria sono uguali, anche se sono oggetti distinti, e `Arrays.equals` su due `Complex[]` deve funzionare.

Le regole

- un numero complesso ha parte reale e parte immaginaria, `double`; dopo la creazione non cambia più (immutabile)
- somma e prodotto creano un numero nuovo: $(a + bi) + (c + di) = (a + c) + (b + d)i$ e $(a + bi)(c + di) = (ac - bd) + (ad + bc)i$; il modulo è $\sqrt{a^2 + b^2}$
- si stampa come `3.0 + 4.0i`; se la parte immaginaria è negativa, `1.0 - 2.0i`
- `equals(Object)` e `hashCode` coerenti: numeri uguali, hash uguali

La classe `Complex` e il programma `ProvaComplex` sono già scritti. Li leggiamo riga per riga, cercando i tre metodi di `Object` ridefiniti e i quattro passi di `equals`.



Commentiamo insieme – `Complex.java` (1/2)

```java
import java.util.Objects;

/** Un numero complesso, immutabile. */
public class Complex {

    private final double re;
    private final double im;

    public Complex(double re, double im) {
        this.re = re;
        this.im = im;
    }

    public double getRe() {
        return re;
    }

    public double getIm() {
        return im;
    }
```

```java
    /** CREA la somma di this e z. */
    public Complex somma(Complex z) {
        return new Complex(re + z.re,
                           im + z.im);
    }

    /** CREA il prodotto di this e z. */
    public Complex prodotto(Complex z) {
        double a = re * z.re - im * z.im;
        double b = re * z.im + im * z.re;
        return new Complex(a, b);
    }

    public double modulo() {
        return Math.sqrt(re * re + im * im);
    }
```

- una classe immutabile come nella lezione 6: tutto `private final`, nessun `set`, le operazioni **creano** un nuovo `Complex`. `z.re` è lecito: siamo dentro `Complex`
- `import java.util.Objects` serve a `hashCode`, nella prossima slide

Commentiamo insieme – `Complex.java` (2/2)

```java
@Override
public String toString() {
    if (im < 0) {
        return re + " - " + (-im) + "i";
    }
    return re + " + " + im + "i";
}

@Override
public int hashCode() {
    return Objects.hash(re, im);
}
```

- `re + " - "`: un double concatenato a una String diventa testo (lezione 2): `3.0`, non `3`
- `Objects.hash` accetta argomenti di qualunque tipo (i double vengono inscatolati, lezione 4), sugli stessi due campi di `equals`. VS Code sa generare entrambi: *Source Action...* → *Generate hashCode() and equals()*

```java
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    Complex c = (Complex) obj;
    return Double.compare(re, c.re) == 0
            && Double.compare(im, c.im) == 0;
}
```

- i quattro passi: stesso oggetto; `null`; classe diversa; cast e confronto dei campi
- per i `double` il confronto standard è `Double.compare(a, b) == 0`: fra `0.0` e `-0.0`, `==` dice `true` ma gli hash sono diversi e la regola salterebbe; `Double.compare` li distingue, come `Objects.hash`. Per gli `int` basta `==`

Commentiamo insieme – ProvaComplex.java

```
Complex z = new Complex(3, 4);
Complex w = new Complex(3, 4);
Complex u = new Complex(1, -2);

System.out.println(z);
System.out.println(z.somma(u));
System.out.println(z.prodotto(u));

System.out.println(z == w);
System.out.println(z.equals(w));
System.out.println(z.hashCode() == w.hashCode());
System.out.println(z.equals(u));
System.out.println(z.equals("3.0 + 4.0i"));
System.out.println(z.equals(null));

Complex[] a = { z, u };
Complex[] b = { w, new Complex(1, -2) };
System.out.println(Arrays.equals(a, b));
```

```
3.0 + 4.0i
4.0 + 2.0i
11.0 - 2.0i
false
true
true
false
false
false
true
```

Rispondiamo insieme:

1. perché `z == w` è `false` e `z.equals(w)` è `true`?
2. senza `toString`, che cosa stamperebbe la seconda riga?
3. `z.equals("3.0 + 4.0i")` dà `false`: quale passo decide? E con `null`?
4. con un `equals(Complex)` come nella lezione 6 (solo il confronto dei campi) al posto di `equals(Object)`, quali righe cambierebbero?
5. un `hashCode` sempre 0 rispetta la regola? Che cosa si perde?

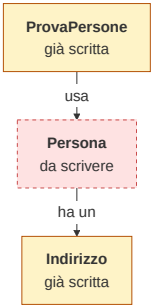
Insieme, passo passo (30 min) – il problema

Il problema. Un'anagrafe tiene le persone con il loro indirizzo: via e città. Una persona può **trasferirsi**, cambiando città. Serve poter **copiare** una persona in modo che la copia sia indipendente: se la copia si trasferisce, l'originale resta dov'è.

Le regole

- una persona ha nome e cognome, che non cambiano, e un indirizzo
- l'indirizzo è *parte* della persona: nasce con lei, dai dati passati al costruttore, e non esce mai dalla classe (composizione). Di fuori si legge solo la città
- una persona si stampa come `Anna Russo, via Roma 1, Napoli`
- due persone sono uguali se hanno nome, cognome e indirizzo uguali; `hashCode` coerente
- `clone()` restituisce una copia **profonda**: con il proprio indirizzo

`Indirizzo` e `ProvaPersone` sono già scritti: leggiamo prima il programma di prova, poi `Indirizzo`, e scriviamo `Persona`.



`Persona` è client di `Indirizzo` ed è usata da `ProvaPersone`: i due ruoli insieme, come nella lezione 5.

Insieme, passo passo – ProvaPersone.java

```
public static void main(String[] args)
        throws CloneNotSupportedException {
    Persona p = new Persona("Anna", "Russo",
            "via Roma 1", "Napoli");
    Persona q = p.clone();
    System.out.println(p);
    System.out.println(q);
    System.out.println(p == q);
    System.out.println(p.equals(q));

    q.trasferisci("Milano");
    System.out.println(p);
    System.out.println(q);
    System.out.println(p.equals(q));
    System.out.println(p.getCitta());
}
```

Deve stampare:

```
Anna Russo, via Roma 1, Napoli
Anna Russo, via Roma 1, Napoli
false
true
Anna Russo, via Roma 1, Napoli
Anna Russo, via Roma 1, Milano
false
Napoli
```

Come sempre, il client viene prima della classe: leggendolo ricaviamo l'**interfaccia** di `Persona`: il costruttore con quattro `String`, `clone`, `toString` (implicito in `println`), `equals`, `trasferisci`, `getCitta`.

Due indizi: dopo `q.trasferisci("Milano")` la città di `p` è ancora Napoli, quindi i due indirizzi sono oggetti **distinti**; e `p.equals(q)` passa da `true` a `false`, quindi `equals` guarda anche l'indirizzo.

Il `main` dichiara `throws CloneNotSupportedException` perché chiama `clone`.

Insieme, passo passo – `Indirizzo.java` (già scritta)

```java
public class Indirizzo implements Cloneable {

    private final String via;
    private String citta;

    public Indirizzo(String via,
                     String citta) {
        this.via = via;
        this.citta = citta;
    }

    @Override
    public String toString() {
        return via + ", " + citta;
    }

    @Override
    public int hashCode() {
        return Objects.hash(via, citta);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        Indirizzo altro = (Indirizzo) obj;
        return via.equals(altro.via)
                && citta.equals(altro.citta);
    }

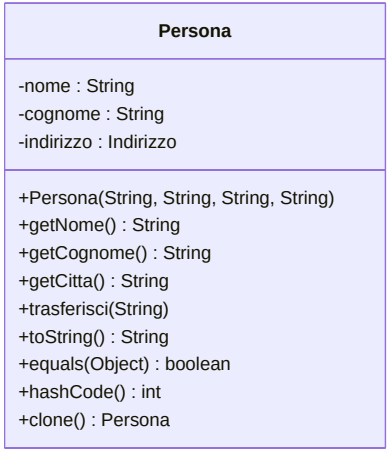
    @Override
    public Indirizzo clone()
            throws CloneNotSupportedException {
        return (Indirizzo) super.clone();
    }
}
```

Non mostrati: `setCitta(String)`, i due `get`, l'`import` di `Objects`. `clone` è superficiale, e basta: contiene solo `String`.

Insieme, passo passo – dal programma di prova alla classe

Prima di scrivere codice, decidiamo insieme:

1. quali **variabili di istanza**? → `nome` e `cognome`, `private final`; `indirizzo`, `private` ma **non** `final`: `clone` dovrà sostituirlo nella copia
2. il **costruttore**? → riceve quattro `String` e crea lui l'`Indirizzo`: composizione, nessun riferimento esterno da copiare
3. che cosa espone dell'indirizzo? → solo `getCitta()`, che delega a `indirizzo.getCitta()`. Nessun `getIndirizzo`: restituirebbe il riferimento interno
4. `trasferisci`? → delega: `indirizzo.setCitta(citta)`
5. `toString`? → `nome + " " + cognome + ", " + indirizzo`: la concatenazione chiama il `toString` di `Indirizzo`
6. `equals`? → i quattro passi; nell'ultimo anche `indirizzo.equals(altra.indirizzo)`, che è quello di `Indirizzo`. `hashCode` con `Objects.hash(nome, cognome, indirizzo)`
7. `clone`? → `implements Cloneable`; `super.clone()` per la copia superficiale, poi `c.indirizzo = indirizzo.clone()` per renderla profonda



Una possibile soluzione (1/2) – Persona.java

```java
/** Una persona e il suo indirizzo. */
public class Persona implements Cloneable {

    private final String nome;
    private final String cognome;
    private Indirizzo indirizzo;   // non final

    public Persona(String nome, String cognome,
                   String via, String citta) {
        this.nome = nome;
        this.cognome = cognome;
        indirizzo = new Indirizzo(via, citta);
    }

    public String getNome() {
        return nome;
    }

    public String getCognome() {
        return cognome;
    }

    public String getCitta() {
        return indirizzo.getCitta();
    }

    /** MODIFICA: cambia citta'. */
    public void trasferisci(String citta) {
        indirizzo.setCitta(citta);
    }

    @Override
    public String toString() {
        return nome + " " + cognome
                + ", " + indirizzo;
    }
```

- `new Indirizzo(via, citta)` nel costruttore: l'indirizzo nasce con la persona e nessun altro ne ha il riferimento (`import` di `Objects` non mostrato)
- `getCitta` e `trasferisci` **delegano** all'indirizzo: il client parla con la persona, come con l'auto
- `", " + indirizzo`: Java chiama `indirizzo.toString()`

Una possibile soluzione (2/2) – Persona.java

```java
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    Persona altra = (Persona) obj;
    return nome.equals(altra.nome)
            && cognome.equals(altra.cognome)
            && indirizzo.equals(altra.indirizzo);
}
```

- `indirizzo.equals(altra.indirizzo)` e `Objects.hash(..., indirizzo)` usano `equals` e `hashCode` di `Indirizzo`: se non li ridefinisse, due persone con indirizzi uguali ma distinti sarebbero diverse

```java
@Override
public int hashCode() {
    return Objects.hash(nome, cognome,
            indirizzo);
}

@Override
public Persona clone()
        throws CloneNotSupportedException {
    // 1. copia superficiale
    Persona c = (Persona) super.clone();
    // 2. profonda: anche l'indirizzo
    c.indirizzo = indirizzo.clone();
    return c;
}
```

- dopo `super.clone()`, `c.indirizzo` porta allo **stesso** `Indirizzo` di `this`: il passo 2 lo sostituisce con una copia (`c.indirizzo` è lecito: siamo dentro `Persona`). `nome` e `cognome` restano condivisi: `String`, immutabili
- senza il passo 2, `ProvaPersone` stamperebbe Milano anche per `p`, e l'ultimo `equals` darebbe `true`

Riepilogo: cosa abbiamo trattato

Concetto (unità)	**Dove compare oggi**
`Object`, radice di ogni classe, e i suoi metodi (U15)	`ProvaObject`: le versioni ereditate
`toString` (U15)	`Persona` versione 4, `Complex` e tutte le classi di oggi
`equals(Object)`: overload o ridefinizione, i quattro passi (U15)	`Persona`, `Studente`, `Complex`, `Indirizzo`
`hashCode` e la sua regola (U15)	`Objects.hash` nelle stesse classi
`getClass`, `instanceof`, downcasting sicuro (U15)	`ProvaInstanceof`; il passo 3 di `equals`
`clone`, `Cloneable`, copia superficiale e profonda (U15, U23)	`Persona.clone`; `Persona`–`Indirizzo`
Costruttore di copia o `clone` (U23)	la tabella; `x.clone()` su uno `Studente`
Composizione e aggregazione (U16)	`Auto`–`Motore`, `Persona`–`Indirizzo`; `Biblioteca`–`Libro`
Dipendenza (U16)	`ProvaRelazioni`; `Biblioteca.aggiungi(Libro)`; `Auto.accendi()`
Ereditarietà o contenimento (U16)	`Studente` è-una `Persona`; `Auto` ha-un `Motore`

Prossima lezione (L9): le interfacce – `interface` e `implements` (finalmente `Cloneable`), ereditarietà multipla di tipo, metodi `default` e `static`, interfacce o classi astratte; i package – `package`, `import`, compilazione con le directory, il `protected` della lezione 7 e il riepilogo dei modificatori.

Homework 8 (da svolgere entro la prossima lezione)

Tre esercizi, ciascuno con il suo programma di prova (`ProvaData`, `ProvaAppuntamento`, `ProvaAnagrafe`). Ogni classe ridefinisce `toString`, `equals(Object)` con i quattro passi e `hashCode` con `Objects.hash`, tranne `Anagrafe`, che ridefinisce solo `toString`. Niente `try/catch`: `throws CloneNotSupportedException` dove serve, come oggi.

1. `Data.java`: una data del calendario, immutabile: giorno, mese e anno `int` (precondizione: data valida). Tre `get`; `isPrimaDi(Data altra)`; `toString()` nella forma `03/11/2026`, giorno e mese sempre a due cifre (un metodo `private static` aggiunge lo zero). `ProvaData` crea due date uguali e una diversa e stampa: `==` ed `equals` fra le due uguali, il confronto dei loro `hashCode`, `isPrimaDi` nei due versi, una data concatenata a una stringa, e `Arrays.equals` su due `Data[]` con date uguali ma oggetti distinti.
2. `Appuntamento.java` (con `Orario.java`): `Orario` modificabile, ore e minuti, con `aggiungiMinuti(int n)` ($n \geq 0$; passata la mezzanotte si riparte da `00:00`), `toString` nella forma `09:30` e `clone()` superficiale. `Appuntamento` ha un titolo e un `Orario` che crea nel costruttore `Appuntamento(String titolo, int ore, int minuti)` (composizione); `getOre()` e `getMinuti()` delegano all'orario; `posticipa(int n)`; `toString` come `Ricevimento alle 09:30`; `clone()` con copia **profonda**. `ProvaAppuntamento` clona un appuntamento, posticipa solo la copia e stampa entrambi ed `equals`, prima e dopo.
3. `Anagrafe.java`: copiate `Persona` e `Studente` (versione 4) in `homework/`. `Anagrafe` tiene un `Persona[]`, con la capienza data al costruttore, e quante persone contiene (aggregazione: le persone nascono fuori). `aggiungi(Persona p)`: `false` se è piena o se c'è già una persona uguale a `p`; `contiene(Persona p)`, con `equals`; `quantiStudenti()`, con `instanceof`; `toString()` con una persona per riga. `ProvaAnagrafe` aggiunge una persona, uno studente, una persona uguale alla prima (rifiutata) e uno studente con il nome e il cognome della prima (accettato: perché?); stampa un `contiene`, il numero di persone, gli studenti e l'anagrafe.

Domande?

mattia.fonisto@unina.it